МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Практикум №1-2

з курсу «Аналіз даних в інформаційнних системах»

на тему: «Підключення до джерел даних. Створення сховища даних»

Викладач:

Олійник Ю.О.

Виконала:

студентка 2 курсу

групи ІП-44 ФІОТ

Бідюк М.А.

Київ-2026

1. ХАРАКТЕРИСТИКА ТА ОПИС ДАНИХ

Для виконання роботи було проведено аналіз доступних відкритих даних та відібрано три ключові набори, які дозволяють комплексно оцінити стан транспортної сфери України.

1. Реєстр транспортних засобів (МВС України)

Найбільш об'ємним та інформативним джерелом є державний реєстр операцій з транспортними засобами, який охоплює повний період з 2022 по 2024 роки. Файл містить деталізовану історію реєстраційних дій, де кожен рядок відповідає окремій операції. Структура файлу дозволяє виконувати глибоку аналітику за кількома напрямками:

- **Адміністративно-територіальний аналіз:** наявність поля REG_ADDR_KOATUU є критично важливою, оскільки воно містить уніфікований код території реєстрації. Це дозволяє уникнути проблем з різним написанням назв областей та міст, забезпечуючи точну прив'язку до виміру географії.
- **Суб'єкти та операції:** поле PERSON розмежовує фізичних (P) та юридичних (J) осіб, що важливо для сегментації ринку. Атрибути OPER_CODE, OPER_NAME та D_REG (дата реєстрації) дають змогу відслідковувати динаміку ринку в часі та популярність різних типів послуг (первинна реєстрація, перереєстрація тощо).
- **Технічні характеристики автопарку:** дані містять вичерпний опис об'єкта: ідентифікацію (BRAND, MODEL, VIN, MAKE_YEAR), фізичні параметри (COLOR, OWN_WEIGHT, TOTAL_WEIGHT) та експлуатаційні характеристики (FUEL, CAPACITY). Поля KIND, BODY та PURPOSE дозволяють класифікувати транспорт за призначенням, що необхідно для виокремлення комерційного та приватного сектору.

2. Аварійно-небезпечні ділянки (Державне агентство відновлення та розвитку інфраструктури України)

Дане джерело представляє собою аналітичний реєстр місць концентрації ДТП, сформований станом на 2024 рік. Ключовою особливістю набору даних є

те, що він містить вже агреговані показники аварійності, накопичені за попередній трирічний період. Це дозволяє використовувати інформацію не як простий перелік окремих інцидентів, а як карту ризиків дорожньої мережі України.

З точки зору архітектури сховища, оскільки вихідний набір даних містить агреговані показники, а не деталізовані події, відповідна таблиця фактів (fact_road_safety) класифікується як факти періодичного зрізу. Для збереження цілісності моделі galaxy schema, атрибут date_key заповнюється датою публікації звіту (1 січня відповідного року). Такий підхід дозволяє коректно відображати стан аварійності у розрізі років публікації статистики, не порушуючи зв'язків із загальним календарем dim_date.

- **Геопросторова прив'язка:** набір містить високоточні дані локалізації. Поля GPS-координати (широта/довгота для початку та кінця ділянки) дозволяють візуалізувати небезпечні зони на карті. Атрибути Індекс дороги (наприклад, М-06) та КМ+м (кілометрова прив'язка) забезпечують інтеграцію з іншими інфраструктурними даними. Поле Область слугує зовнішнім ключем для зв'язку з виміром Dim_Location.
- **Класифікація небезпеки:** ключовими метриками є розрахунковий Коефіцієнт пригод та категоризований ступінь небезпеки. Ці атрибути є основою для побудови виміру Dim_Danger_Class та дозволяють сегментувати дороги за рівнем ризику для водіїв.
- **Історичний контекст:** поле Кількість ДТП за три роки до взяття на облік є готовим фактом для аналізу. Воно показує інтенсивність аварійності, яка призвела до включення ділянки у список небезпечних. Це дозволяє аналізувати накопичений ефект небезпеки, а не лише миттєві події.

3. Транспортні засоби, що перебувають у розшуку (Нацполіція)

Третім джерелом обрано офіційний набір даних про незаконні заволодіння транспортними засобами. Дані охоплюють широкий часовий діапазон з 2017 по 2023 роки, що дозволяє виявити довгострокові тенденції криміногенної ситуації та сезонність злочинів.

- **Інформаційна цінність:** інтеграція цього джерела додає до сховища вимір «Кримінальна безпека». Це дозволяє розрахувати метрики ризику для власників авто, співставляючи популярність моделі з реєстру з її популярністю серед викрадачів.
- **Ключові атрибути:**
  - **Ідентифікація об'єкта (brandmodel, cartype, color):** атрибути бренду та моделі є ключовими для зв'язку з відповідними нормалізованими довідниками (dim_brand, dim_model) та центральною таблицею фактів автопарку (fact_vehicle). Це дозволяє визначити найбільш «ризиковані» марки та моделі, враховуючи їх історичні зміни. Поле color додає деталізацію через зв'язок з dim_color.
  - **Унікальні ідентифікатори (vehiclenumber, bodynumber, chassisnumber, enginenumber):** наявність номерів кузова, шасі та двигуна дозволяє (теоретично) перевіряти перетин з основним реєстром: чи не був зареєстрований автомобіль, що знаходиться в розшуку (або навпаки – чи не був викрадений щойно зареєстрований ТЗ).
  - **Часовий та географічний контекст (illegalseizuredate, organunit):** дата незаконного заволодіння (illegalseizuredate) є основою для фактів у часовому вимірі. Поле organunit (назва підрозділу поліції, що оголосив розшук) виступає джерелом для визначення регіону події та прив'язки до виміру Dim_Location.
  - **Службові дані (insertdate):** дата внесення інформації в базу, що дозволяє оцінити оперативність роботи служб (різниця між датою крадіжки та датою фіксації).

4. Офіційний класифікатор КОАТУУ (Державна служба статистики України)

Для розв'язання проблеми відсутності текстових назв населених пунктів у первинних реєстрах МВС, до сховища інтегровано додаткове джерело – класифікатор об'єктів адміністративно-територіального устрою України (КОАТУУ). Файл містить ієрархічну структуру кодів (розбиту на чотири рівні),

категорію населеного пункту (місто, село, СМТ) та його офіційну назву українською мовою. Використання цього довідника є необхідним для подальшого збагачення географічного виміру та забезпечення можливості побудови деталізованих геопросторових візуалізацій.

2. АРХІТЕКТУРА ТА МОДЕЛІ ДАНИХ

Для реалізації сховища обрано архітектуру Galaxy Schema. Це дозволяє об'єднати три незалежні бізнес-процеси (реєстрація, аварійність, викрадення) через спільні виміри.

2.1. Модель Stage зони

Для забезпечення надійності імпорту всі поля визначені з запасом по довжині або як текстові, щоб уникнути помилок типізації на етапі завантаження.

Нижче наведено DDL-скрипти для створення таблиць Stage зони у СКБД PostgreSQL:

```
CREATE TABLE staging.stg_vehicle_registry (
  person_type VARCHAR(50),
  koatuu_code VARCHAR(50),
  oper_code INT,
  oper_name VARCHAR(255),
  reg_date VARCHAR(50),
  dep_code INT,
  dep_name VARCHAR(255),
  brand VARCHAR(100),
  model VARCHAR(100),
  vin VARCHAR(100),
  make_year INT,
  color VARCHAR(100),
  kind VARCHAR(100),
  body_type VARCHAR(100),
  purpose VARCHAR(100),
  fuel_type VARCHAR(100),
```

```
    capacity VARCHAR(50),
    own_weight VARCHAR(50),
    total_weight VARCHAR(50),
    n_reg_new VARCHAR(50)
);

CREATE TABLE staging.stg_road_blackspots (
    region_name VARCHAR(150),
    road_index VARCHAR(50),
    road_name VARCHAR(255),
    km_start VARCHAR(50),
    km_end VARCHAR(50),
    gps_start VARCHAR(100),
    gps_end VARCHAR(100),
    accident_coeff VARCHAR(50),
    danger_level VARCHAR(50),
    accidents_3years INT
);

CREATE TABLE staging.stg_wanted_vehicles (
    brand_model VARCHAR(255),
    car_type VARCHAR(100),
    color VARCHAR(100),
    vehicle_number VARCHAR(50),
    body_number VARCHAR(100),
    chassis_number VARCHAR(100),
    engine_number VARCHAR(100),
    seizure_date VARCHAR(50),
    organ_unit VARCHAR(255),
    insert_date VARCHAR(50)
);
```

```
CREATE TABLE staging.stg_koatuu_dict (
    level1 VARCHAR(20),
    level2 VARCHAR(20),
    level3 VARCHAR(20),
    level4 VARCHAR(20),
    category VARCHAR(5),
    name_ua VARCHAR(255)
);
```

2.2. Модель основного сховища

Основне сховище проєктується у схемі core. Воно складається з нормалізованих таблиць вимірів (dimensions) та таблиць фактів (facts), що забезпечують аналітичну гнучкість та швидкість виконання запитів.

2.2.1. Dimensions

Виміри містять описові атрибути та слугують для фільтрації, групування та зрізу даних.

```
CREATE TABLE core.dim_date (
    date_key INT PRIMARY KEY,
    full_date DATE, year INT, quarter INT, month INT, day_of_week INT
);

CREATE TABLE core.dim_location (
    location_key SERIAL PRIMARY KEY,
    koatuu_code VARCHAR(20) UNIQUE,
    oblast_name VARCHAR(100),
    district_name VARCHAR(100),
    settlement_type VARCHAR(50),
    settlement_name VARCHAR(100),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8)
```

```
);

CREATE TABLE core.dim_brand (
    brand_key SERIAL PRIMARY KEY,
    brand_name VARCHAR(100),
    valid_from DATE NOT NULL,
    valid_to DATE,
    is_current BOOLEAN DEFAULT TRUE
);

CREATE TABLE core.dim_model (
    model_key SERIAL PRIMARY KEY,
    model_name VARCHAR(100)
);

CREATE TABLE core.dim_color (
    color_key SERIAL PRIMARY KEY,
    color_name VARCHAR(100) UNIQUE
);

CREATE TABLE core.dim_operation (
    oper_key SERIAL PRIMARY KEY,
    oper_code INT,
    oper_name VARCHAR(255) UNIQUE
);

CREATE TABLE core.dim_road_risk (
    risk_key SERIAL PRIMARY KEY,
    road_index VARCHAR(50),
    road_name VARCHAR(255),
```

```
    danger_level VARCHAR(50)
);
```

Для забезпечення можливості побудови геопросторових візуалізацій на картах та BI-звітів з ієрархічним заглибленням, код КОАТУУ декомпозовано у таблиці dim_location. Вимір заздалегідь спроєктовано з атрибутами для області, району, типу та назви населеного пункту. Передбачається, що ці атрибути будуть заповнюватися не лише з первинних транзакційних систем, але й шляхом мапінгу з еталонним довідником stg_koatuu_dict для відновлення повної ієрархії.

dim_brand спроєктовано з використанням SCD Type 2. Наявність полів valid_from та is_current дозволяє коректно фіксувати ребрендинг компаній-виробників, зберігаючи історичну правду для минулих подій.

2.2.2. Facts

Таблиці фактів містять кількісні показники (метрики) та зовнішні ключі на відповідні виміри.

```
CREATE TABLE core.fact_vehicle (
    vehicle_key SERIAL PRIMARY KEY,
    vin VARCHAR(100) UNIQUE,
    date_key INT REFERENCES core.dim_date(date_key),
    brand_key INT REFERENCES core.dim_brand(brand_key),
    model_key INT REFERENCES core.dim_model(model_key),
    color_key INT REFERENCES core.dim_color(color_key),
    make_year INT,
    fuel_type VARCHAR(100),
    vehicle_type VARCHAR(100)
);

CREATE TABLE core.fact_registrations (
    fact_id SERIAL PRIMARY KEY,
    date_key INT REFERENCES core.dim_date(date_key),
```

```
    location_key INT REFERENCES core.dim_location(location_key),
    vehicle_key INT REFERENCES core.fact_vehicle(vehicle_key),
    oper_key INT REFERENCES core.dim_operation(oper_key),
    operation_count INT DEFAULT 1,
    engine_capacity INT,
    car_weight DECIMAL(10,2)
);

CREATE TABLE core.fact_road_safety (
    fact_id SERIAL PRIMARY KEY,
    date_key INT REFERENCES core.dim_date(date_key),
    location_key INT REFERENCES core.dim_location(location_key),
    risk_key INT REFERENCES core.dim_road_risk(risk_key),
    accident_count INT, danger_coeff DECIMAL(10,2)
);

CREATE TABLE core.fact_thefts (
    fact_id SERIAL PRIMARY KEY,
    date_key INT REFERENCES core.dim_date(date_key),
    location_key INT REFERENCES core.dim_location(location_key),
    vehicle_key INT REFERENCES core.fact_vehicle(vehicle_key),
    theft_count INT DEFAULT 1
);
```

2.3. Логічна схема (ER-діаграма)

На наведеній нижче схемі зображено архітектуру «galaxy schema». Особливістю моделі є трансформація сутності автомобіля у профільну таблицю фактів (fact_vehicle), що посилається на деталізовані довідники (dim_brand, dim_model). Це вирішує проблему великої кількість унікальних транспортних засобів у реєстрі та дозволяє прив'язати життєвий цикл кожного авто до загального виміру часу (dim_date).

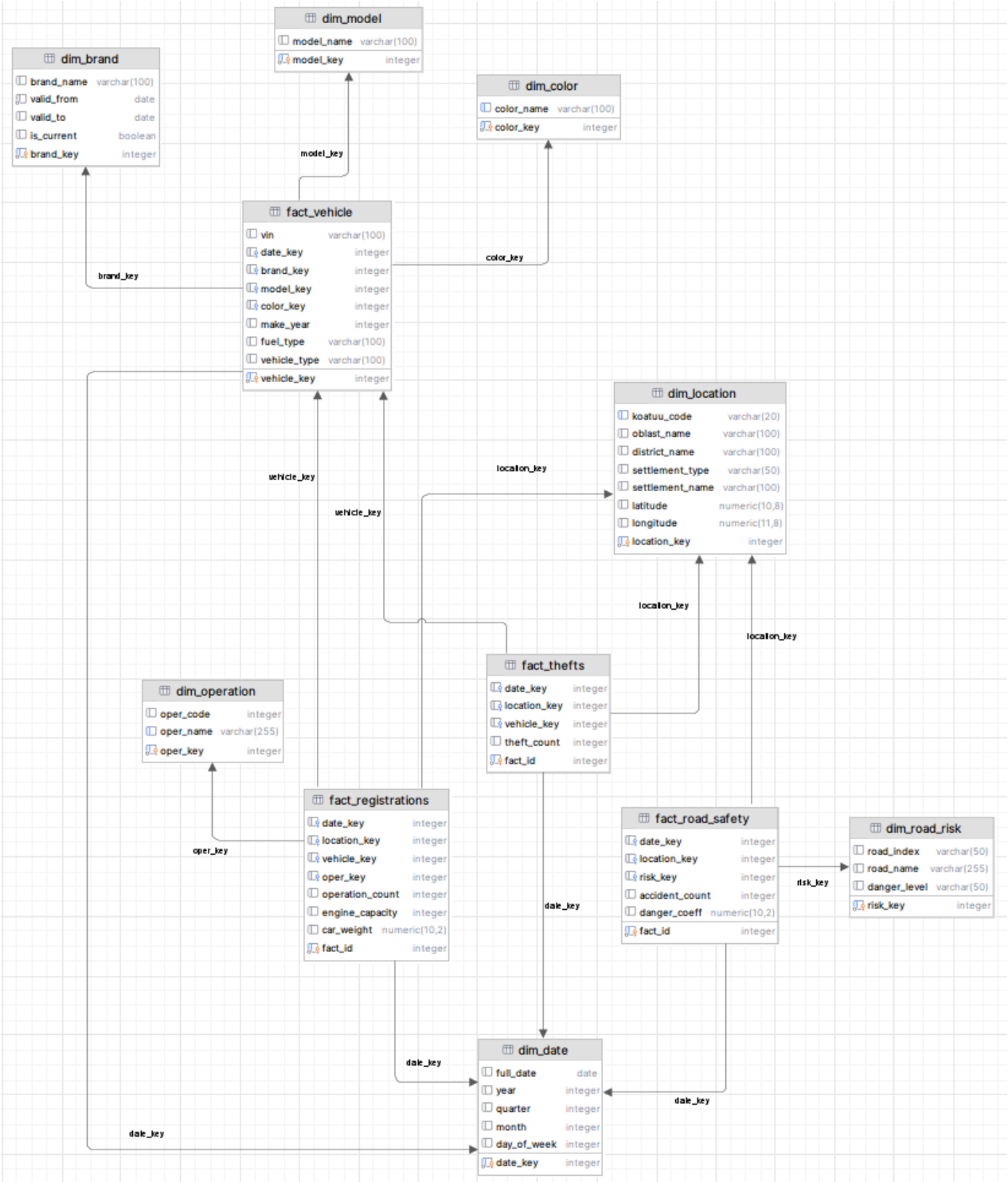

Рисунок 2.1 – ER-діаграма спроєктованого сховища даних

3. ETL-ПРОЦЕСИ

Для наповнення спроєктованого сховища даних розроблено набір ETL-процедур засобами СКБД PostgreSQL. Процес поділено на логічні етапи: екстракція у буферну зону, очищення та збагачення вимірів, завантаження фактів.

3.1. Завантаження даних до stage-зони

Первинне завантаження «сирих» даних (реєстр МВС, ДТП, розшук) здійснюється у транзитні таблиці схеми staging у текстовому форматі (VARCHAR), що гарантує збереження даних без втрат через конфлікти типів. Також до stage-зони було завантажено офіційний Державний класифікатор КОАТУУ (stg_koatuu_dict) для подальшого збагачення географічних даних.

3.2. Трансформація, очищення та збагачення

Під час перенесення даних зі Stage до Core зони реалізовано наступні механізми Data Quality:

- **Очищення (Cleansing):** Застосовано функції TRIM() для видалення прихованих пробілів та UPPER() для нормалізації текстових значень (уникнення дублікатів брендів/моделей через різний регістр). Заміна ком на крапки у числових полях здійснена через REPLACE.
- **Вирішення аномалій дат:** Виявлено проблему некоректних форматів року у сирих даних МВС (наприклад, двозначний рік «23» замість «2023»). Імплементовано динамічний парсинг: якщо згенерований date_key менший за 1 000 000, до нього математично додається константа для перетворення у формат YYYYMMDD. Для оптимізації обчислень на великих обсягах даних (понад 6 млн записів) логіку парсингу винесено у загальний табличний вираз (CTE). Це дозволило виконувати ресурсомістку функцію TO_CHAR(TO_DATE(...)) лише один раз для кожного рядка, уникаючи багаторазового переобчислення у конструкції CASE.
- **Збагачення даних (Data Enrichment):** Код КОАТУУ декомпозовано на складові (область, район). Для відновлення текстових назв населених пунктів використано злиття (JOIN) таблиці локацій із завантаженим довідником КОАТУУ. Це дозволило успішно замінити NULL-значення на реальні назви міст та сіл.

3.3. Стратегія завантаження

Використано конструкцію ON CONFLICT DO NOTHING як для вимірів, так і для таблиці фактів (fact_vehicle). Відмова від використання підзапитів з NOT IN дозволила уникнути проблеми Nested Loop та кардинально зменшити час виконання ETL-процедур при обробці мільйонів рядків.

Для довідника брендів (dim_brand) реалізовано збереження історії змін.

3.4. Оптимізація продуктивності (індексування)

Оскільки обсяг первинних даних у реєстрі транспортних засобів перевищує 6 мільйонів записів, критичним етапом проєктування стала оптимізація продуктивності бази даних. Для прискорення виконання операцій об'єднання (JOIN), фільтрації та пошуку дублікатів під час роботи ETL-процедур, було розроблено стратегію індексування.

На таблицях Stage-зони та Core-сховища створено некластеризовані B-Tree індекси. Зокрема, реалізовано функціональний індекс (UPPER(TRIM(oper_name))) для прискорення нормалізації текстових даних "на льоту" без навантаження на процесор під час виконання запиту.

```
CREATE INDEX IF NOT EXISTS idx_stg_vin ON
staging.stg_vehicle_registry(vin);
CREATE INDEX IF NOT EXISTS idx_stg_koatuu ON
staging.stg_vehicle_registry(koatuu_code);
CREATE INDEX IF NOT EXISTS idx_stg_oper_func ON
staging.stg_vehicle_registry (UPPER(TRIM(oper_name)));

CREATE INDEX IF NOT EXISTS idx_fact_veh_vin ON core.fact_vehicle(vin);
CREATE INDEX IF NOT EXISTS idx_dim_oper_name ON
core.dim_operation(oper_name);
```

3.5. SQL-скрипти ETL-процедур

```
CREATE OR REPLACE PROCEDURE core.etl_load_dim_date(start_date DATE,
end_date DATE)
```

```
LANGUAGE plpgsql AS $$
BEGIN
   INSERT INTO core.dim_date (date_key, full_date, year, quarter, month,
day_of_week)
   SELECT
      TO_CHAR(datum, 'YYYYMMDD')::INT AS date_key,
      datum AS full_date,
      EXTRACT(YEAR FROM datum) AS year,
      EXTRACT(QUARTER FROM datum) AS quarter,
      EXTRACT(MONTH FROM datum) AS month,
      EXTRACT(ISODOW FROM datum) AS day_of_week
   FROM (SELECT generate_series(start_date, end_date, '1 day'::interval) AS
datum) AS d
   ON CONFLICT (date_key) DO NOTHING;
END;
$$;

CREATE OR REPLACE PROCEDURE core.etl_load_dimensions()
LANGUAGE plpgsql AS $$
BEGIN
   INSERT INTO core.dim_location (koatuu_code, oblast_name, district_name)
   SELECT DISTINCT
      koatuu_code,
      COALESCE(dep_name, 'Невідома область') AS oblast_name,
      CASE
         WHEN LENGTH(koatuu_code) >= 5 THEN SUBSTRING(koatuu_code, 1, 5)
|| ' Район'
         ELSE 'Невідомо'
      END AS district_name
   FROM staging.stg_vehicle_registry
   WHERE koatuu_code IS NOT NULL
```

```
ON CONFLICT (koatuu_code) DO NOTHING;

UPDATE core.dim_location dim
SET
    settlement_name = dict.name_ua,
    settlement_type = CASE
        WHEN dict.category = 'M' THEN 'Місто'
        WHEN dict.category = 'C' THEN 'Село'
        WHEN dict.category = 'T' THEN 'СМТ'
        WHEN dict.category = 'Щ' THEN 'Селище'
        WHEN dict.category = 'P' THEN 'Район міста'
        ELSE dim.settlement_type
    END
FROM staging.stg_koatuu_dict dict
WHERE TRIM(dim.koatuu_code) = TRIM(COALESCE(dict.level4, dict.level3,
dict.level2, dict.level1));

INSERT INTO core.dim_brand (brand_name, valid_from, is_current)
SELECT DISTINCT
    UPPER(TRIM(brand)),
    CURRENT_DATE,
    TRUE
FROM staging.stg_vehicle_registry
WHERE brand IS NOT NULL
  AND UPPER(TRIM(brand)) NOT IN (SELECT brand_name FROM
core.dim_brand);

INSERT INTO core.dim_color (color_name)
SELECT DISTINCT UPPER(TRIM(COALESCE(color, 'НЕВІДОМО')))
FROM staging.stg_vehicle_registry
ON CONFLICT (color_name) DO NOTHING;
```

```
    INSERT INTO core.dim_model (model_name)
    SELECT DISTINCT UPPER(TRIM(COALESCE(model, 'НЕВІДОМО')))
    FROM staging.stg_vehicle_registry
    WHERE model IS NOT NULL
    ON CONFLICT DO NOTHING;

    INSERT INTO core.dim_operation (oper_code, oper_name)
    SELECT DISTINCT
        oper_code,
        UPPER(TRIM(COALESCE(oper_name, 'НЕВІДОМА ОПЕРАЦІЯ')))
    FROM staging.stg_vehicle_registry
    WHERE oper_name IS NOT NULL
    ON CONFLICT (oper_name) DO NOTHING;
END;
$$;

CREATE OR REPLACE PROCEDURE core.etl_load_facts()
LANGUAGE plpgsql AS $$
BEGIN
    WITH parsed_data AS (
        SELECT
            vin,
            UPPER(TRIM(brand)) AS brand_clean,
            UPPER(TRIM(model)) AS model_clean,
            UPPER(TRIM(color)) AS color_clean,
            make_year::INT,
            UPPER(TRIM(fuel_type)) AS fuel_type_clean,
            TO_CHAR(TO_DATE(reg_date, 'DD.MM.YYYY'), 'YYYYMMDD')::INT
AS raw_date_key
        FROM staging.stg_vehicle_registry
```

```
        WHERE reg_date IS NOT NULL
    ),
    deduplicated_vehicles AS (
        SELECT DISTINCT ON (vin)
            COALESCE(vin, 'UNKNOWN_' || md5(random()::text)) AS vin,
            CASE
                WHEN raw_date_key < 1000000 THEN raw_date_key + 20000000
                ELSE raw_date_key
            END AS date_key,
            brand_clean,
            model_clean,
            color_clean,
            make_year,
            fuel_type_clean
        FROM parsed_data
    )
    INSERT INTO core.fact_vehicle (vin, date_key, brand_key, model_key,
color_key, make_year, fuel_type)
    SELECT
        dv.vin,
        dv.date_key,
        b.brand_key,
        m.model_key,
        c.color_key,
        dv.make_year,
        dv.fuel_type_clean
    FROM deduplicated_vehicles dv
    LEFT JOIN core.dim_brand b ON dv.brand_clean = b.brand_name AND
b.is_current = TRUE
    LEFT JOIN core.dim_model m ON dv.model_clean = m.model_name
    LEFT JOIN core.dim_color c ON dv.color_clean = c.color_name
```

```
ON CONFLICT (vin) DO NOTHING;

WITH parsed_registrations AS (
    SELECT
        vin,
        koatuu_code,
        UPPER(TRIM(oper_name)) AS oper_name_clean,
        CAST(REPLACE(total_weight, ',', '.') AS DECIMAL(10,2)) AS car_weight,
        TO_CHAR(TO_DATE(reg_date, 'DD.MM.YYYY'), 'YYYYMMDD')::INT
AS raw_date_key
    FROM staging.stg_vehicle_registry
    WHERE reg_date IS NOT NULL
)
INSERT INTO core.fact_registrations (date_key, location_key, vehicle_key,
oper_key, operation_count, car_weight)
SELECT
    CASE
        WHEN pr.raw_date_key < 1000000 THEN pr.raw_date_key + 20000000
        ELSE pr.raw_date_key
    END AS date_key,
    l.location_key,
    v.vehicle_key,
    o.oper_key,
    1 AS operation_count,
    pr.car_weight
FROM parsed_registrations pr
LEFT JOIN core.dim_location l ON pr.koatuu_code = l.koatuu_code
LEFT JOIN core.fact_vehicle v ON pr.vin = v.vin
LEFT JOIN core.dim_operation o ON pr.oper_name_clean = o.oper_name;
```

END;
$$;

3.6. Результати завантаження даних до основного сховища

Після успішної компіляції та виклику розроблених ETL-процедур (команди CALL core.etl_load_dimensions(); та CALL core.etl_load_facts();), таблиці основного сховища в схемі core були автоматично наповнені, очищені та збагачені.

Для перевірки коректності відпрацювання механізму SCD Type 2 (збереження історії повільно змінюваних вимірів), виконано SQL-запит до таблиці брендів:

```
SELECT brand_key, brand_name, valid_from, valid_to, is_current
FROM core.dim_brand
LIMIT 10;
```

Результат виконання запиту (Рисунок 3.1) підтверджує, що для нових записів автоматично проставляється дата початку дії (valid_from) та прапорець актуальності (is_current = true).

	brand_key	brand_name	valid_from	valid_to	is_current
1	1	RHON	2026-02-22	<null>	true
2	2	GNIOTPOL	2026-02-22	<null>	true
3	3	WEFERLINGEN	2026-02-22	<null>	true
4	4	HOENKHAUS	2026-02-22	<null>	true
5	5	GOEBEL	2026-02-22	<null>	true
6	6	TROUILLET	2026-02-22	<null>	true
7	7	ELLINGHAUS	2026-02-22	<null>	true
8	8	WILK	2026-02-22	<null>	true
9	9	KRONE	2026-02-22	<null>	true
10	10	A3XM	2026-02-22	<null>	true

Рисунок 3.1 – Фрагмент заповненого виміру брендів (dim_brand) із реалізацією SCD Type 2

Для перевірки результатів збагачення даних географічного виміру, виконано SQL-запит до таблиці локацій. Запит демонструє записи, де первинні коди КОАТУУ були успішно перетворені на текстові назви населених пунктів за допомогою інтегрованого довідника:

```
SELECT koatuu_code, oblast_name, settlement_type, settlement_name
FROM core.dim_location
WHERE settlement_name IS NOT NULL
ORDER BY random()
LIMIT 15;
```

Результат виконання (Рисунок 3.2) підтверджує успішне злиття сирих даних МВС із зовнішнім словником (Master Data), що усунуло проблему NULL-значень для валідних кодів і підготувало вимір до геопросторового аналізу у BI-системах.

	koatuu_code	oblast_name	settlement_type	settlement_name
1	6320410100	Харківська область	Місто	БАРВІНКОВЕ
2	5621888702	Рівненська область	Село	БРОДЕЦЬ
3	5322084007	Полтавська область	Село	КАЛАШНИКИ
4	6323982504	Харківська область	Село	МИРОЛЮБІВКА
5	5320881903	Полтавська область	Село	ЛУТАЙКА
6	5324285701	Полтавська область	Село	ШИЛІВКА
7	1221882201	Дніпропетровська область	Село	ГРУЗЬКЕ
8	4625580602	Львівська область	Село	ДНІСТРИК-ДУБОВИЙ
9	5123383807	Одеська область	Село	ШАЙТАНКА
10	5125485902	Одеська область	Село	ВАЛЕНТИНІВКА
11	6321786004	Харківська область	Село	ДУДКІВКА
12	5320685401	Полтавська область	Село	МАНЖЕЛІЯ
13	4410345307	Луганська область	Селище	ХРИСТОФОРІВКА
14	7324080501	Чернівецька область	Село	ОЛЕКСІЇВКА
15	5322081604	Полтавська область	Село	КИСЕЛІВКА

Рисунок 3.2 – Фрагмент виміру локацій (dim_location) після збагачення даними з довідника КОАТУУ

Висновок: У ході виконання практикумів було успішно спроєктовано та наповнено багатовимірне сховище даних для комплексного аналізу транспортної системи України на базі СКБД PostgreSQL. Проаналізовано структуру відкритих державних наборів даних (реєстр МВС, статистика ДТП, база розшуку Нацполіції та класифікатор КОАТУУ), на основі яких розроблено логічну архітектуру типу «Galaxy schema». Реалізовано фізичну модель даних, що включає транзитну буферну Stage-зону та нормалізоване Core-сховище.

Розроблено повноцінний ETL-конвеєр засобами PL/pgSQL-процедур. У процесі перенесення даних імплементовано механізми забезпечення якості: автоматичне очищення текстових атрибутів, розв'язання проблеми аномалій

форматів дат та нормалізація числових показників. Успішно реалізовано збагачення географічного виміру текстовими назвами населених пунктів через інтеграцію з еталонним довідником КОАТУУ. Для повільно змінюваних вимірів (зокрема, довідника брендів) налаштовано відстеження історичних змін за методологією SCD Type 2.

Особливу увагу в роботі приділено оптимізації продуктивності бази даних для ефективної обробки великих масивів інформації (понад 6 мільйонів транзакційних записів реєстру). Для прискорення процесів пошуку, об'єднання та нормалізації було розроблено та впроваджено стратегію індексування. Крім того, логіку ETL-процедур оптимізовано за допомогою загальних табличних виразів (CTE) та заміни ресурсомістких фільтрів NOT IN на механізм ON CONFLICT DO NOTHING. Це дозволило уникнути проблеми неефективних вкладених циклів, мінімізувати навантаження на процесор та радикально скоротити час наповнення таблиць фактів.